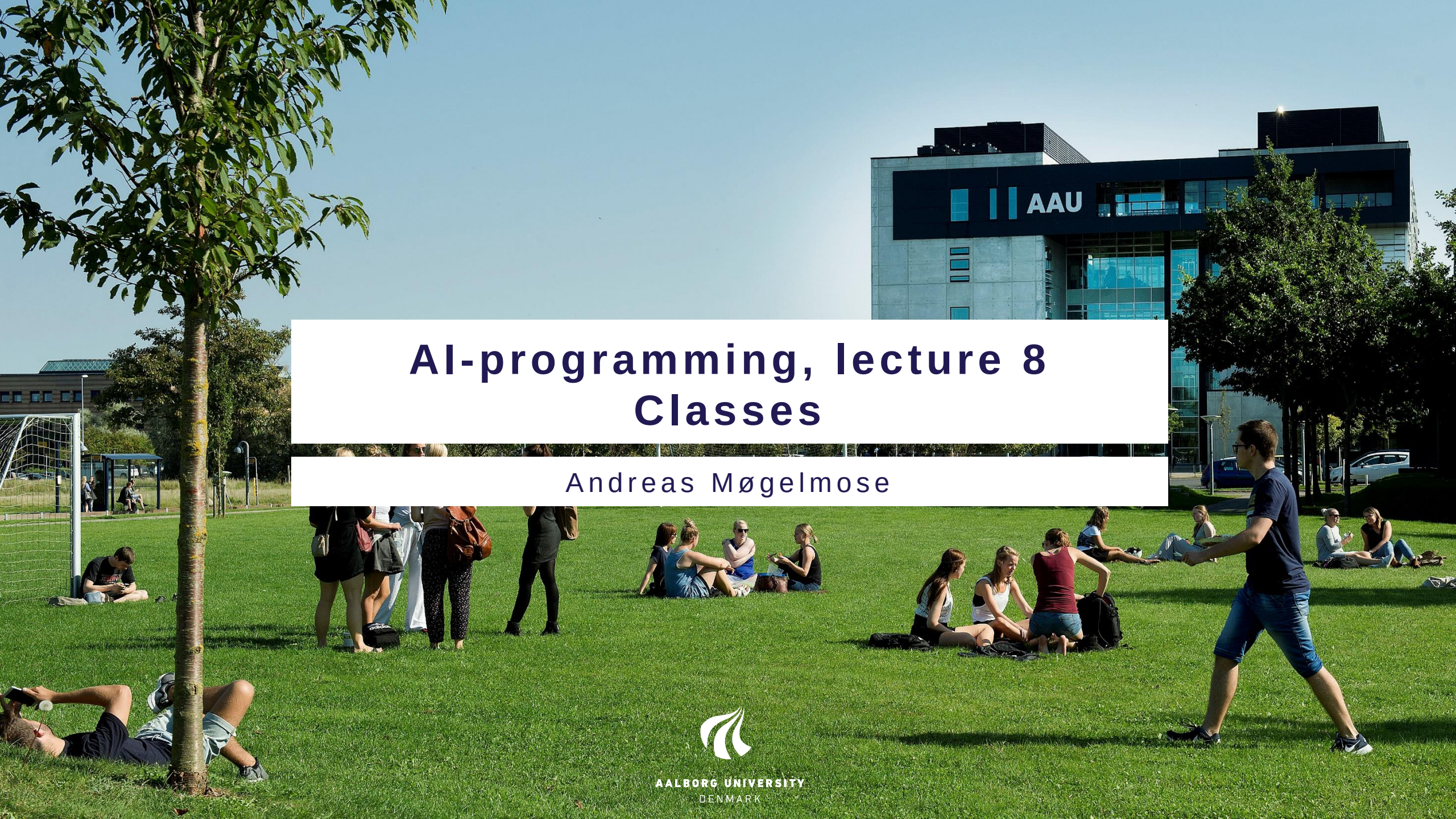


AI-programming, lecture 8
Classes

Andreas Møgelmose

AALBORG UNIVERSITY
DENMARK

Agenda

- Classes and objects
- Attributes and methods
- Inheritance



Recap: Functions

```
my_string.lower()

move(7, 2)

QuitProgram()
```



Recap: Variables and lists

```
name = 'Bob'
age = 21
print(f"My name is {name}. I am {age} years old.")

print(20 % 7)

vegetables = ['carrot', 'lettuce', 'onion', 'radish']
vegetables.append('potato')
```



Recap: Loops

```
cars = ['audi', 'bmw', 'subaru', 'toyota']
cars = [car.title() for car in cars]

for car in cars[1:3]:
    print(car)
```



Recap: Branching

```
while True:
    screen.fill((100, 200, 100))
    s = datetime.now().second
    if s > 30 and enable_rect:
        pygame.draw.rect(screen, (200, 100, 100), (30, 70, 140, 60))
        if s == 59:
            enable_rect = False
    else:
        pygame.draw.ellipse(screen, (100, 100, 200), (30, 70, 140, 60))
```



Recap: Dicts/while/input

```
ages = {"Alice": 22, "Bob": 27, "Carol": 47}
name = input("Enter your name: ")
age = int(input("Enter your age: "))
ages[name] = age
print(ages)
```



Recap: Defining functions

```python
def car_info(number_of_cars, cars):
    number_of_cars += 1
    cars.append("Toyota")
    return number_of_cars


number_of_cars = 3
cars = ["Audi", "BMW", "Ford"]

number_of_cars = car_info(number_of_cars, cars)

print(number_of_cars)
print(cars)
```

Agenda

- Classes and objects
- Attributes and methods
- Inheritance



Object-orientation

What is a car?

How to turn a car into code?



Object-orientation















Object-orientation

- Observations
  - Cars vary, but they are still cars
  - We can identify objects in the real world that are cars

- Conclusion
  - All cars must have some common, defining traits
  - A car is an object that has all the car traits

Object-orientation

- What are the car traits?
  - 4 wheels
  - Engine
  - Doors
  - Seats
  - Transports people and/or goods
  - Needs a driver



Example

```
def main():
    pygame.init()

    screen = pygame.display.set_mode((640, 480))

    car = Car()
    print(f"Car is at: {car.position}")

    running = True
    while running:
        screen.fill((255, 255, 255))

        car.draw(screen)

        pygame.display.flip()
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
```

```
class Car:
    def __init__(self):
        self.position = (100, 100)
        self.color = (255, 0, 0)
        self.size = (20, 30)

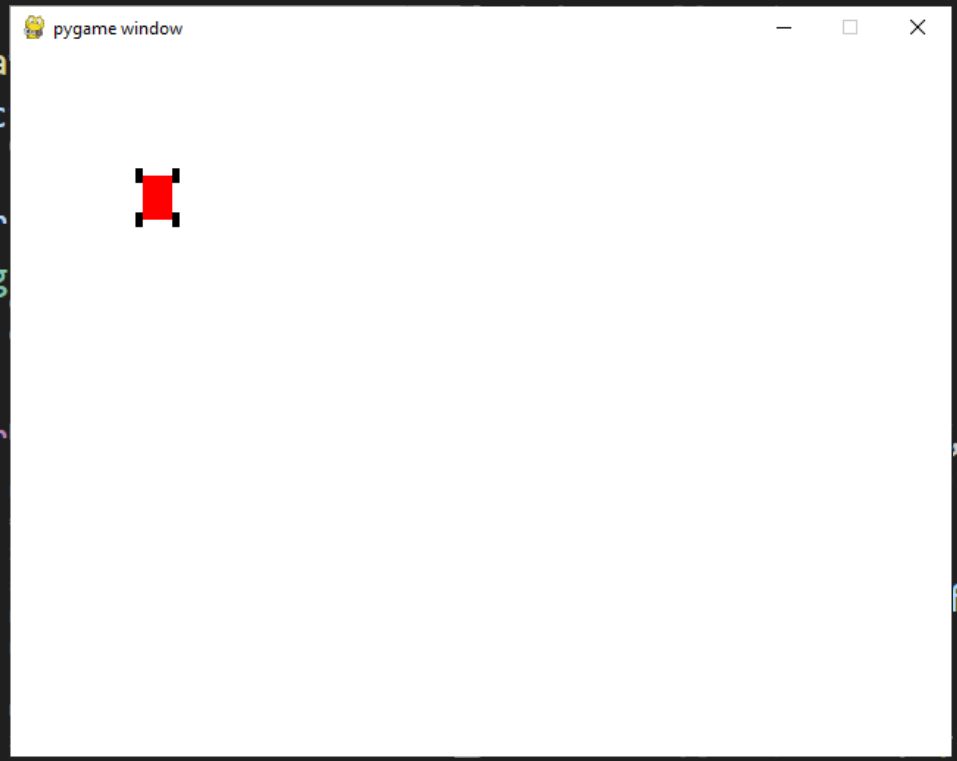
    def dra
        rec

        tir
        pyg

        w,
        for                                    , -h/2),

                                                f.positio

    def drive(self, new_position):
        self.position = new_position
```



Classes & objects

- Lets you write code to match real world concepts
- Allows you to define and manipulate many similar objects in a consistent way
- Allows simpler reuse of code
- Allows you to make clear specifications of your design before implementation
- Allows you to organize your programs



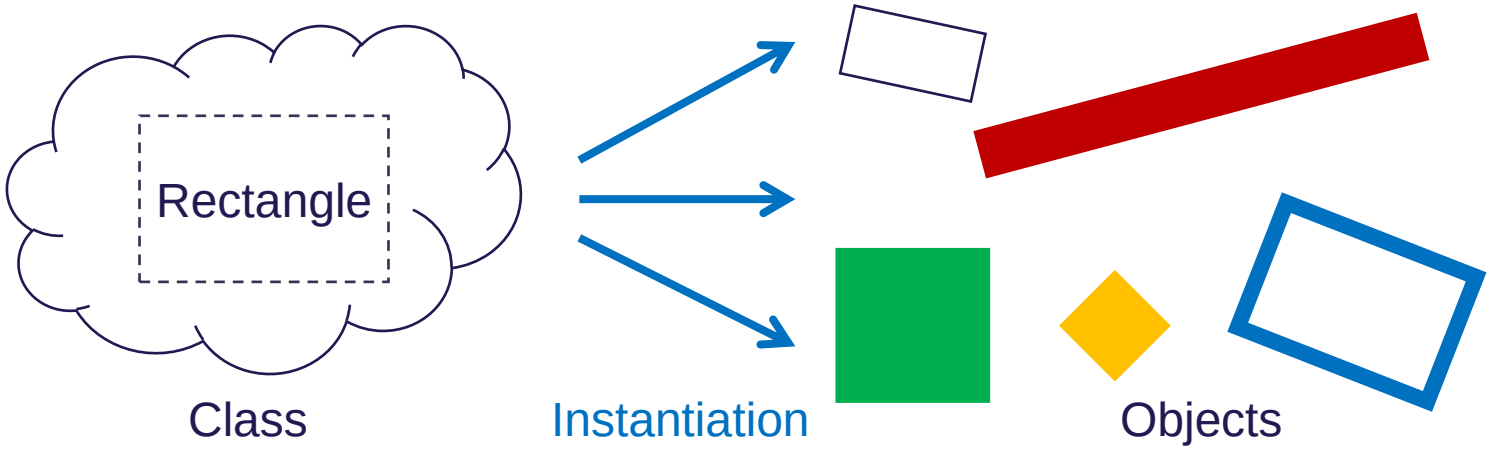
Classes & objects

- Let you write code to match real world concepts
- Allow you to define and manipulate many similar objects in a consistent way
- Allow simpler reuse of code
- Allow you to make clear specifications of your design before implementation
- **Allow you to organize your programs**



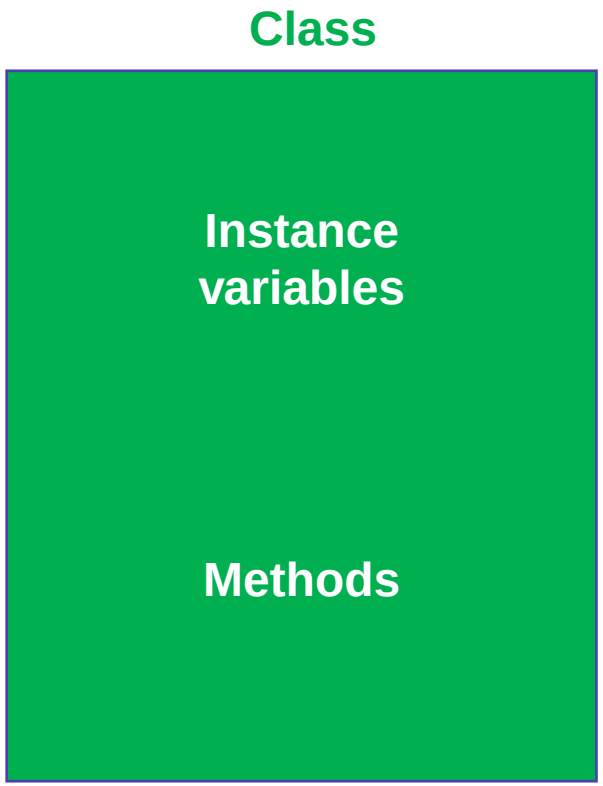
Classes & objects

- The general concept is called a class
- The specific instance of the concept is called an object
- Creating an object of a given class is called instantiation



Class contents

- Variables
  - Called **attributes**, instance variables, properties, fields, or data members
- Functions
  - Called **methods** or member functions



A class is simply a collection of instance variables and methods

Example in details

```python
class Car:
    def __init__(self, starting_position):
        self.position = starting_position
        self.color = (255, 0, 0)
        self.size = (20, 30)

    def draw(self, screen): ...

    def drive(self, new_position):
        self.position = new_position
```

```python
car = Car((50, 50))
print(f"Car is at: {car.position}")
car.drive((200, 200))
car.position = (300, 300)
```



Inheritance

```
dumper = Truck((100, 100))
dumper.drive((400, 400))
dumper.offload()
```

```
class Car:
    def __init__(self, starting_position):
        self.position = starting_position
        self.color = (255, 0, 0)
        self.size = (20, 30)

    def draw(self, screen):...

    def drive(self, new_position):
        self.position = new_position
```

```
class Truck(Car):
    def __init__(self, starting_position):
        super().__init__(starting_position)
        self.capacity = 10000

    def offload(self):
        print("Offloading cargo...")

    def draw(self):...
```

Composition

```python
class Truck(Car):
    def __init__(self, starting_position):
        super().__init__(starting_position)
        self.cargo = Box(1000)
        self.crane = Crane()
        self.snowplow = None

    def offload(self):
        print("Offloading cargo...")

    def draw(self):...
```



Modules

module_import.py

```
1   import my_module
2
3   my_module.module_function()
```

module_import.py

```
1   from my_module import module_function
2
3   module_function()
```

my_module.py > ...

```
1   def module_function():
2       print("Yay, module_function!")
3
```



This could easily be a class





Exercise time!

AALBORG UNIVERSITY
DENMARK